

Avaliação da Capacidade de LLMs na Geração de User Stories: Um Estudo Comparativo com o Framework QUS

Diego Araújo Teixeira¹

¹ Instituto Federal de Educação, Ciência e Tecnologia do Sudeste de Minas Gerais
(IF Sudeste MG) – Campus Manhuaçu
Rodovia BR-116, Km 589,8 (Distrito de Realeza) – 36.909-300 – Manhuaçu – MG – Brazil

1. Objetivo

1.1. Objetivo Geral

Avaliar a capacidade de Modelos de Linguagem de Grande Escala (LLMs) em gerar histórias de usuário, comparando a qualidade sintática e pragmática das histórias geradas por diferentes modelos a partir de descrições de sistemas reais, com base no framework QUS (Quality User Story).

1.2. Objetivos Específicos

- Investigar em que medida LLMs conseguem gerar histórias de usuário aderentes aos critérios de qualidade do framework QUS a partir de descrições informais de sistemas.
- Comparar o desempenho de três LLMs distintas (Claude, DeepSeek e Gemini) quanto à conformidade sintática, atonicidade e minimalidade das histórias geradas.
- Identificar padrões recorrentes de defeitos por modelo, de modo a caracterizar o comportamento típico de cada LLM diante de uma mesma tarefa de elicitação.
- Discutir as implicações desses resultados para o uso de LLMs como ferramentas de apoio em processos ágeis de levantamento de requisitos.

2. Materiais e Métodos

A cronograma deste estudo seguiu quatro etapas principais: seleção da base de dados, geração das histórias de usuário via LLMs, avaliação automatizada por meio da ferramenta AQUASA-core, e consolidação dos resultados.¹

2.1. Seleção da base de dados

Como fonte de descrições de sistemas, utilizou-se o dataset público TAWOS (The Agile World Open Source dataset)[Tawosi et al. 2022], que reúne issues reais extraídas de projetos ágeis open source hospedados no GitHub. O dataset foi importado localmente em um banco de dados relacional MySQL, a partir do qual foram extraídas as descrições por meio de consultas SQL às tabelas Project e Component, conforme apresentado na Figura 1. Essas tabelas fornecem visões gerais e resumos executivos de sistemas e módulos, sem conter histórias de usuário previamente escritas.

¹Todos os artefatos produzidos neste trabalho estão disponíveis em: <https://github.com/diegoaraujoteixeira/llm-user-stories-qus>

Foram selecionadas 20 descrições distintas, priorizando registros com descrição não nula e com no mínimo 50 caracteres, de modo a garantir contexto suficiente para a tarefa de elicitación.

```
-- Para descrições de projetos inteiros:
SELECT Name, Description
FROM Project
WHERE Description IS NOT NULL AND LENGTH(Description) > 50
LIMIT 20;

-- Para descrições de módulos mais específicos:
SELECT p.Name as ProjectName, c.Name as ComponentName, c.Description
FROM Component c
JOIN Project p ON c.Project_ID = p.ID
WHERE c.Description IS NOT NULL AND LENGTH(c.Description) > 50
LIMIT 20;
```

Figura 1. Consultas SQL utilizadas para extração das descrições do dataset TAWOS

2.2. Geração das histórias de usuário

Cada uma das 20 descrições foi submetida a três LLMs (Claude, DeepSeek e Gemini) por meio de um prompt padronizado, estruturado segundo a técnica de zero-shot prompting, na qual não são fornecidos exemplos de saída, apenas a instrução da tarefa. O prompt utilizado seguiu o modelo apresentado na Figura 2, no qual os campos Project, Component e Description eram substituídos, a cada execução, pelos valores extraídos do dataset TAWOS correspondentes ao cenário avaliado.

```
Task: Write 3 to 5 user stories for the '<Component>'
in the <Project> project.
```

```
Context:
Project: <Project>
Component: <Component>
Description: <Description>.
```

```
Please create a list of user stories based on these
features so the team knows what to build. List them
one per line.
```

Figura 2. Prompt padronizado utilizado na geração das histórias de usuário

As saídas foram coletadas em inglês, já que a ferramenta de avaliação utilizada processa exclusivamente textos nesse idioma.

2.3. Avaliação automatizada

As histórias geradas por cada modelo foram compiladas em arquivos de texto plano (.txt) e submetidas à ferramenta AQUA-core (Automatic Quality User Story Analyzer), de-

envolvida pelo Requirements Engineering Lab da Utrecht University com base no framework QUS proposto por [Lucassen et al. 2016]. A ferramenta realiza processamento de linguagem natural (tokenização e part-of-speech tagging) para identificar violações automaticamente em três dimensões de qualidade propostas pelo framework QUS:

- Sintática: se a história segue a estrutura padrão e mantém uniformidade gramatical em relação às demais histórias do mesmo conjunto (critérios Well-formed e Uniform);
- Semântica: se a história utiliza termos ambíguos ou vagos que dificultam sua testabilidade (critério Ambiguous);
- Pragmática: se a história representa uma única funcionalidade, sem conectivos que indiquem múltiplas ações (critério Atomic), e se está livre de detalhes redundantes ou exemplos técnicos entre parênteses (critério Minimal).

Nos conjuntos analisados, não foram identificadas violações na dimensão semântica, concentrando-se as ocorrências nos critérios sintáticos e pragmáticos.

Vale ressaltar que a escolha do AQUISA como avaliador, em vez de uma avaliação manual, teve como objetivo eliminar viés do pesquisador e garantir reprodutibilidade total do processo.

2.4. Consolidação dos resultados

Os relatórios HTML gerados pelo AQUISA para os 60 conjuntos de histórias (20 descrições × 3 modelos) foram tabulados manualmente, contabilizando o tipo e a frequência de cada defeito por modelo. A partir dessa tabulação, calculou-se a densidade média de defeitos por história e o número de conjuntos sem nenhuma violação, permitindo uma comparação quantitativa e qualitativa entre os três LLMs avaliados.

Todos os artefatos produzidos (descrições de entrada, prompt utilizado, saídas brutas das LLMs e relatórios do AQUISA) estão disponíveis publicamente no seguinte repositório: <https://github.com/diegoaraujoteixeira/llm-user-stories-qus>, referenciando o dataset [Tawosi et al. 2022] e o repositório oficial do AQUISA-core [Dalpiaz and Lucassen 2016].

3. Resultados

Esta seção apresenta os resultados quantitativos e qualitativos obtidos a partir da análise das histórias de usuário geradas pelos três LLMs avaliados (Claude, DeepSeek e Gemini), com base nos relatórios produzidos pela ferramenta AQUISA-core para os 20 cenários extraídos do dataset TAWOS.

3.1. Análise Quantitativa Geral

A Tabela 1 apresenta uma visão consolidada do desempenho de cada modelo. Foram geradas 100 histórias de usuário por LLM (5 histórias em média por cenário, ao longo de 20 cenários), totalizando 300 histórias analisadas.

Tabela 1. Resultados quantitativos gerais por modelo

Modelo	Cenários sem defeitos	Ocorrências totais	Média por cenário
Gemini	1/20 (5%)	44	2,20
Claude	0/20 (0%)	68	3,40
DeepSeek	0/20 (0%)	167	8,35

Os dados evidenciam uma diferença expressiva entre os modelos: o DeepSeek apresentou um volume de violações quase quatro vezes maior que o Gemini e mais que o dobro do Claude, enquanto o Gemini foi o único modelo a produzir um conjunto de histórias totalmente livre de defeitos (cenário referente ao módulo Stream, do projeto Spring XD).

3.2. Distribuição dos Defeitos por Critério QUS

A Tabela 2 detalha o percentual de cenários (sobre os 20 avaliados) em que cada modelo apresentou ao menos uma violação por critério do framework QUS.

Tabela 2. Percentual de cenários com violação por critério QUS

Critério	Gemini	Claude	DeepSeek
Atomic (conjunções)	75%	55%	100%
Minimal (parênteses)	30%	100%	100%
Uniform (sintaxe)	40%	35%	50%

3.3. Perfil de Desempenho por Modelo

Gemini. Foi o modelo com menor média de defeitos (2,20 por cenário) e o único a produzir um conjunto de histórias integralmente conforme ao QUS. Seu ponto fraco concentrou-se no critério de atomicidade, presente em 75% dos cenários, geralmente pela combinação de duas ações relacionadas na mesma história, como em:

“As a data architect, I want to implement processing modules that filter and transform messages in real-time to ensure data quality before storage.”

Em contrapartida, o Gemini apresentou a menor taxa de violações de minimalidade (30%), demonstrando maior em evitar detalhes técnicos.

Claude. Apresentou desempenho intermediário (3,40 defeitos/cenário), com comportamento oposto ao do Gemini, onde 100% dos cenários apresentaram ao menos uma violação de minimalidade, geralmente pela inserção sistemática de exemplos técnicos entre parênteses, como em:

“As a data engineer, I want to configure a source module (e.g., HTTP, file, JMS) so that I can ingest data into a stream from various external systems.”

Por outro lado, o Claude foi o modelo com menor incidência de falhas de atomicidade (55%), sugerindo maior tendência a manter uma única ação por história, ainda que às custas de explicações adicionais entre parênteses.

DeepSeek. Foi o modelo com pior desempenho global, apresentando violações de atomicidade e minimalidade em 100% dos cenários avaliados, além de ser o único modelo a apresentar o critério de pontuação (*punctuation*) como defeito adicional. O padrão observado nas saídas do DeepSeek indica uma tendência à sobre-especificação técnica, combinando múltiplos exemplos de implementação na mesma história, como em:

“As a developer, I want to create a source module that ingests data from external systems (e.g., file, HTTP, TCP, or JMS) so that I can bring streaming data into the Spring XD pipeline.”

Esse comportamento é consistente ao longo de todo o conjunto de dados: nenhum dos 20 cenários gerados pelo DeepSeek ficou livre de pelo menos uma violação simultânea de atomicidade e minimalidade.

4. Discussão

4.1. Convergência Sintática e Divergência Pragmática

Os resultados indicam que, mesmo sem qualquer restrição explícita de formato no prompt (Seção 2.2), os três modelos convergiram naturalmente para uma estrutura sintática próxima ao padrão QUS (“As a..., I want..., so that...”), o que explica a baixa incidência do critério *Uniform* (35% a 50%) em comparação aos critérios pragmáticos *Atomic* e *Minimal* (55% a 100%). Isso sugere que a principal dificuldade das LLMs avaliadas não está na adoção da sintaxe de uma user story, mas na granularidade da funcionalidade descrita e no nível de detalhe técnico incluído nas respostas.

É provável que o grau de liberdade concedido pelo prompt utilizado, que não impôs restrições explícitas de atomicidade ou minimalidade, tenha contribuído para a alta incidência desses dois critérios. Esse estudo, no entanto, não avaliou comparativamente um prompt com restrições mais rígidas (por exemplo, proibindo explicitamente o uso de conectivos ou de exemplos entre parênteses), de modo que essa relação entre engenharia de prompt e conformidade QUS permanece como uma hipótese a ser verificada em trabalhos futuros.

4.2. Comparação com a Literatura

[Lucassen et al. 2016] demonstraram, na validação original do AQUASA sobre 1.023 histórias escritas por profissionais de 18 empresas, que falhas de atomicidade e minimalidade também figuram entre os defeitos mais recorrentes em histórias escritas por humanos, o que indica que essas dificuldades não são exclusivas de LLMs, mas inerentes à própria tarefa de elicitação de requisitos em linguagem natural. Os resultados deste estudo corroboram essa observação, mesmo com acesso ao contexto completo do sistema, nenhum dos três modelos avaliados foi capaz de gerar um conjunto de histórias totalmente livre de defeitos pragmáticos, com exceção de um único cenário produzido pelo Gemini.

5. Conclusão

Este trabalho avaliou a capacidade de três LLMs (Claude, DeepSeek e Gemini) em gerar histórias de usuário a partir de descrições de sistemas reais, usando o framework QUS e a ferramenta AQUASA-core como critério objetivo de avaliação.

Os resultados mostraram que os três modelos convergiram naturalmente para a sintaxe padrão de user stories, mesmo sem restrições de formato no prompt, mas falharam recorrentemente nos critérios pragmáticos de atomicidade e minimalidade. O Gemini apresentou o melhor desempenho geral, enquanto o DeepSeek mostrou um padrão claro de sobre-especificação técnica, escrevendo histórias mais próximas de documentação de código do que de requisitos ágeis.

A contribuição deste estudo foi demonstrar que, a partir de descrições mínimas de contexto, os modelos avaliados são capazes de elicitar histórias de usuário estruturalmente válidas, mas com perfis de erro distintos entre si, evidenciando que a escolha do modelo utilizado pode impactar diretamente a qualidade dos requisitos gerados em processos ágeis apoiados por IA.

6. Declaração de Uso de IA

O modelo Claude (Anthropic) foi utilizado nas seguintes frentes: auxílio na definição do tema e do desenho experimental, no apoio técnico na instalação da ferramenta AQUUSA-core em ambiente local e na tabulação e organização dos resultados obtidos a partir dos relatórios do AQUUSA-core.

O autor assume integral responsabilidade pelo conteúdo intelectual e técnico do trabalho, em conformidade com a Portaria CNPq nº 2.664/2026

Referências

- Dalpiaz, F. and Lucassen, G. (2016). AQUUSA-core: Automatic quality user story analyzer. <https://github.com/RELabUU/aqusa-core>. Requirements Engineering Lab, Utrecht University. Accessed: 2026-07-06.
- Lucassen, G., Dalpiaz, F., van der Werf, J. M. E., and Brinkkemper, S. (2016). Improving agile requirements: the quality user story framework and tool. *Requirements engineering*, 21(3):383–403.
- Tawosi, V., Al-Subaihin, A., Moussa, R., and Sarro, F. (2022). A versatile dataset of agile open source software projects. In *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*, pages 707–711.